

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет об исследовательском проекте на тему:
Автономная навигация мобильного робота с использованием Robotic Operating
System: алгоритмы планирования и управления

Выполнил студент:

группы #БПМИ244, 2 курса

Злобин Герман Эдуардович

Принял руководитель проекта:

Дергачев Степан Алексеевич

Штатный преподаватель

Базовая кафедра Федерального исследовательского центра
«Информатика и управление» Российской академии наук

Содержание

1	Введение	5
1.1	Описание области и постановка задачи	5
1.2	Структура работы	5
2	Обзор литературы	6
2.1	Обзор методов глобального планирования	6
2.1.1	Алгоритмы поиска по графу	6
2.1.2	Алгоритмы случайной выборки (Sampling-based)	7
2.1.3	Интеллектуальные бионические алгоритмы	7
2.2	Обзор методов локального планирования	8
2.2.1	Классические контроллеры	8
2.2.2	Геометрические или реактивные методы	9
2.2.3	Оптимизационные или предсказательные методы	10
2.3	Сравнительный анализ и выбор алгоритмов	11
2.3.1	Обоснование выбора алгоритма D* Lite	11
2.4	Выводы по главе	11
3	Экспериментальное сравнение алгоритмов A* и D* Lite	14
3.1	Общие концепции работы D* Lite и поведение A* в динамике	14
3.2	Модель и условия эксперимента	15
3.3	Результаты экспериментов	15
3.4	Анализ производительности в статической среде	16
3.5	Анализ производительности в динамической среде	17
3.6	Выводы по главе	17
4	Экспериментальное сравнение локальных планировщиков MPPI и RA-MPPI	19
4.1	Общие концепции работы MPPI и RA-MPPI	19
4.2	Модель и условия эксперимента	20
4.3	Результаты экспериментов	20
4.4	Анализ производительности в статической среде	21
4.5	Анализ производительности в динамической среде	22
4.6	Выводы по главе	22
5	Реализация навигационного стека в ROS 2 и сквозное тестирование	24
5.1	Архитектура и используемые компоненты	24
5.2	Разработанные программные пакеты	24
5.2.1	Реализация D* Lite в виде плагина Nav2	24
5.2.2	Реализация RA-MPPI в виде плагина Nav2	25
5.2.3	Скрипты построения и анализа экспериментов	25

5.3	Среда тестирования и сценарии	26
5.4	Возникшие проблемы и принятые решения	27
5.4.1	Проблемы интеграции Nav2	27
5.4.2	Адаптация алгоритмических компонентов	28
5.4.3	Физика динамических препятствий	28
5.5	Линейное предсказание траекторий динамических препятствий	29
5.6	Результаты экспериментов	30
5.7	Выводы по главе	32
Заключение		33
Список литературы		34

Аннотация

Работа посвящена планированию движения автономного робота в динамической среде. Проведено сравнительное исследование алгоритмов глобального планирования A^* и D^* Lite, а также локальных контроллеров МРПИ и RA-МРПИ. На основе этих результатов D^* Lite и RA-МРПИ реализованы в виде собственных плагинов Nav2, а A^* и МРПИ задействованы через стандартные плагины Nav2; собранный навигационный стек проверен на симуляторе Gazebo Sim с роботом TurtleBot3.

Ключевые слова

Навигация; планирование пути; динамическая среда; мобильные роботы; D^* Lite; A^* ; МРПИ; RA-МРПИ; локальный и глобальный планировщик.

1 Введение

Современная мобильная робототехника активно внедряется в логистику и автоматизацию, где ключевой задачей является обеспечение автономной навигации. Способность робота безопасно и эффективно прокладывать маршрут напрямую определяет его функциональность в реальных условиях.

1.1 Описание области и постановка задачи

Реальные среды эксплуатации (склады, цеха, городская застройка) являются динамическими: положение препятствий и объектов в них заранее неизвестно или постоянно меняется. В таких условиях классические алгоритмы планирования, требующие полной перестройки маршрута при каждом изменении, часто оказываются избыточными и неэффективными.

Целью данной работы является сравнительный анализ алгоритмов планирования и управления движением мобильного робота в динамических средах, а также практическая реализация выбранных алгоритмов в составе индустриального навигационного стека. На уровне глобального планирования рассматривается переход от классического поиска (A^*) к инкрементальным методам (D^* Lite), позволяющим обновлять маршрут без полного пересчёта графа состояний; на уровне локального управления — сравнение стохастических предсказательных контроллеров MPPI и RA-MPPI.

Для достижения цели решаются задачи обзора и классификации методов глобального и локального планирования, обоснования архитектуры системы, разработки программных моделей для измерения времени перепланирования A^*/D^* Lite и качества траекторий MPPI/RA-MPPI, а также интеграции выбранных алгоритмов в виде плагинов Nav2 и сквозного тестирования собранного навигационного стека на физическом симуляторе.

1.2 Структура работы

Работа состоит из введения, четырёх основных глав, заключения и списка литературы. Первая глава посвящена обзору и классификации алгоритмов глобального и локального планирования, а также обоснованию выбора D^* Lite в качестве глобального планировщика. Вторая глава содержит описание методики эксперимента и анализ результатов сравнения A^* и D^* Lite по критериям скорости и стабильности вычислений на крупномасштабных картах. Третья глава посвящена экспериментальному сравнению локальных планировщиков MPPI и RA-MPPI по критериям качества траекторий и успешности навигации в статической и динамической средах. Четвёртая глава описывает практическую реализацию выбранных алгоритмов в виде плагинов Nav2 в составе экосистемы ROS 2 и сквозное тестирование собранного навигационного стека на физическом симуляторе Gazebo Sim с роботом TurtleBot3.

2 Обзор литературы

В современных системах управления автономными роботами задачу навигации принято разделять на два иерархических уровня: глобальное и локальное планирование. Такое разделение обусловлено необходимостью баланса между вычислительной сложностью и скоростью реакции на изменения среды.

Глобальное планирование отвечает за построение оптимального маршрута на всей известной карте. Однако расчет такого пути требует значительных ресурсов и не может выполняться с высокой частотой. Локальное планирование, напротив, работает в ограниченном «окне» видимости сенсоров, обеспечивая безопасный объезд внезапно возникших препятствий и удержание робота на глобальной траектории в режиме реального времени. Совместное использование этих уровней позволяет роботу сохранять целенаправленность движения, не теряя при этом реактивности.

Задаче планирования движения мобильных роботов в динамических средах посвящено большое количество научных исследований. В обзорных работах [1, 2, 3], предлагаются классификации методов планирования, анализируются их достоинства и ограничения. В данной главе рассматриваются основные классы алгоритмов глобального и локального планирования с акцентом на их применимость в условиях изменяющейся среды.

2.1 Обзор методов глобального планирования

Методы глобального планирования предназначены для построения маршрута от начальной точки к цели на основе априорной или частично известной карты среды. Как правило, они ориентированы на поиск оптимального или близкого к оптимальному пути и используются на верхнем уровне навигационной системы.

2.1.1 Алгоритмы поиска по графу

Алгоритмы поиска по графу относятся к классическим методам планирования и широко применяются в задачах навигации мобильных роботов [3]. Они предполагают дискретизацию пространства в виде графа или решётки и поиск пути между вершинами.

Алгоритм Дейкстры [4] предназначен для поиска кратчайшего пути в графе с неотрицательными весами рёбер. Он гарантирует нахождение оптимального решения, однако требует обработки большого числа вершин, что приводит к высокой вычислительной сложности при увеличении размера карты.

Алгоритм A^* [5] является развитием метода Дейкстры и использует эвристическую функцию для ускорения поиска. При корректном выборе эвристики A^* существенно снижает количество рассматриваемых вершин и демонстрирует высокую эффективность в двумерных и слаборазмерных средах. Тем не менее, в сложных и высокоразмерных пространствах его производительность снижается.

В условиях динамической среды классические алгоритмы Дейкстры и A^* требуют полного пересчёта маршрута при изменении информации о препятствиях, что ограничивает

их применение в реальном времени.

Для устранения данного недостатка были разработаны инкрементальные алгоритмы перепланирования. **Алгоритм D*** [6] и его модификация **D* Lite** [7] позволяют обновлять ранее найденный путь при появлении новых данных об окружающей среде. **Алгоритм Lifelong Planning A* (LPA*)** [7] поддерживает повторное использование результатов предыдущего поиска. **Алгоритм ARA*** [8] реализует anytime-подход, обеспечивая быстрое получение приближённого решения с последующим улучшением.

2.1.2 Алгоритмы случайной выборки (Sampling-based)

Алгоритмы случайной выборки (sampling-based planners) предназначены для планирования движения в непрерывных и высокоразмерных пространствах конфигураций [9]. Они основаны на случайном семплировании допустимых состояний и построении графа или дерева достижимых конфигураций.

Алгоритм Rapidly-exploring Random Tree (RRT) [10] осуществляет итеративное расширение дерева в направлении случайных точек пространства. Он быстро находит допустимые траектории в сложных средах и обладает хорошими исследовательскими свойствами. Однако получаемые пути, как правило, далеки от оптимальных.

Алгоритм RRT* [11] является модификацией RRT и гарантирует асимптотическую оптимальность. По мере увеличения числа семплов качество найденного пути улучшается. Недостатком RRT* является высокая вычислительная сложность и медленная сходимость в средах с узкими проходами.

Алгоритм Batch Informed Trees (BIT*) [12] сочетает идеи эвристического поиска и семплирования. Он ограничивает область поиска с использованием эвристики, аналогичной A*, что повышает эффективность планирования. **Алгоритм RABIT*** [13] расширяет BIT* за счёт применения локальной оптимизации, что ускоряет сходимость к качественным решениям.

Для задач с динамическими препятствиями предлагаются специализированные методы, например **Risk-DTRRT** [14], учитывающий вероятность столкновений и уровень риска при построении траектории.

2.1.3 Интеллектуальные бионические алгоритмы

Бионические и интеллектуальные алгоритмы основаны на моделировании коллективного поведения биологических систем и применяются для решения задач оптимизации [15]. В планировании движения они используются для поиска приближённых оптимальных траекторий.

Генетические алгоритмы (GA) [16] представляют траекторию в виде хромосомы и используют операции скрещивания и мутации для поиска оптимального решения. Они обладают высокой гибкостью и способны адаптироваться к изменениям среды, однако требуют значительных вычислительных ресурсов и не гарантируют сходимости за ограниченное время.

Алгоритмы муравьиной колонии (ACO) [17] основаны на коллективном поиске пути с использованием виртуальных феромонов. Они эффективны в задачах поиска кратчайших маршрутов, но чувствительны к настройке параметров и могут медленно адаптироваться к динамическим изменениям.

Алгоритм искусственной пчелиной колонии (ABC) [18] и **алгоритм роя частиц (PSO)** [19] применяются для оптимизации параметрических представлений траектории. Их преимуществами являются простота реализации и возможность параллельной обработки. Основным недостатком является зависимость качества решения от начальной инициализации и параметров.

2.2 Обзор методов локального планирования

Методы локального планирования предназначены для формирования управляющих воздействий в реальном времени на основе текущего состояния робота и информации о ближайшем окружении. В отличие от глобальных планировщиков, которые строят маршрут на всей карте, локальные методы работают в ограниченной области и ориентированы прежде всего на безопасное движение, обход препятствий и следование заданному глобальному пути. Это делает их особенно важными в динамических средах, где присутствуют движущиеся объекты и неопределённость в данных сенсоров. Как правило, локальные планировщики используются совместно с глобальными: глобальный уровень задаёт опорный маршрут, а локальный корректирует движение робота с учётом текущей ситуации.

2.2.1 Классические контроллеры

Классические контроллеры применяются в первую очередь для задачи слежения за заданной траекторией и стабилизации движения робота. Они не выполняют планирование траектории в строгом смысле, однако широко используются как нижний исполнительный уровень в навигационных системах.

PID-контроллер является одним из наиболее простых и распространённых регуляторов. Он формирует управляющее воздействие на основе текущей ошибки, её интегральной и дифференциальной составляющих. В задачах мобильной робототехники PID применяется для управления линейной и угловой скоростью робота с целью достижения заданной позиции или следования траектории [20]. Основными преимуществами PID-регулятора являются простота реализации, низкие вычислительные затраты и высокая частота работы. К недостаткам относится отсутствие учёта динамики окружающей среды и невозможность самостоятельного избегания препятствий, что ограничивает его применение только исполнительным уровнем.

Линейно-квадратичный регулятор (LQR) основан на минимизации квадратичного функционала качества, учитывающего отклонение состояния системы от заданного и величину управляющего воздействия. В работах [21] и [22] показано, что LQR позволяет обеспечить более устойчивое и оптимальное движение мобильного робота по сравнению с

PID, особенно при задаче слежения за траекторией. LQR учитывает модель динамики системы, что повышает точность управления. Его основным недостатком является необходимость линеаризации модели и чувствительность к ошибкам параметров.

Комбинированные схемы PID+LQR используются для объединения простоты PID и оптимальных свойств LQR. В работе [23] показано, что такой подход позволяет повысить устойчивость системы, улучшить качество позиционирования и подавить колебания. Однако гибридные контроллеры требуют более сложной настройки и точного согласования параметров.

В целом классические контроллеры не являются полноценными локальными планировщиками, но выполняют важную роль исполнительного уровня, обеспечивая реализацию траекторий, полученных более высокими уровнями навигационной системы.

2.2.2 Геометрические или реактивные методы

Геометрические или реактивные методы локального планирования принимают решения непосредственно на основе текущих сенсорных данных и геометрии окружающей среды. Они не строят долгосрочные траектории, а выбирают допустимое направление или скорость движения, обеспечивающие безопасность в ближайшем будущем.

Метод Dynamic Window Approach (DWA) был предложен в [24]. Он рассматривает пространство допустимых линейных и угловых скоростей робота и выбирает те из них, которые достижимы с учётом динамических ограничений и не приводят к столкновениям. Далее используется целевая функция, учитывающая приближение к цели, скорость движения и расстояние до препятствий. В работе [25] предложена модификация DWA, ориентированная на динамические препятствия и групповые движения роботов. Основным достоинством DWA является высокая скорость работы и практическая применимость, однако метод подвержен застреванию в локальных минимумах.

Метод Velocity Obstacles (VO) был предложен в [26] и основан на построении множества скоростей, которые приведут к столкновению с препятствием в будущем. Эти скорости исключаются из допустимого пространства управления. Такой подход позволяет учитывать движение препятствий и прогнозировать возможные столкновения. В работе [27] предложено расширение VO, учитывающее как положение, так и скорости объектов. Преимуществом метода является корректная работа в динамических средах, однако он чувствителен к шуму в измерениях.

Метод искусственных потенциальных полей (APF) был предложен Хатибом в [28]. Цель моделируется как источник притяжения, а препятствия — как источники отталкивания, и робот движется по направлению результирующего градиента. В [29] предложена модификация APF для задач навигации в неизвестной среде с динамическими препятствиями. Основными преимуществами APF являются простота и высокая вычислительная эффективность. Существенным недостатком является наличие локальных минимумов, из-за которых робот может застревать.

Метод Vector Field Histogram (VFH) был представлен в [30] и основан на постро-

ении гистограммы плотности препятствий по данным сенсоров. На её основе выбирается направление движения с минимальной опасностью столкновения. В работе [31] предложены модификации VFH, улучшающие точность построения гистограмм и учитывающие динамические препятствия. Метод хорошо работает в реальном времени, однако не обеспечивает глобальной оптимальности траектории.

Геометрические методы отличаются высокой вычислительной эффективностью и широко применяются в реальных робототехнических системах. Их основным недостатком является локальный характер принятия решений, что может приводить к неоптимальному поведению в сложных средах.

2.2.3 Оптимизационные или предсказательные методы

Оптимизационные методы формулируют задачу локального планирования как задачу оптимального управления, в которой необходимо минимизировать функционал качества при наличии динамических и кинематических ограничений.

Model Predictive Control (MPC) является одним из наиболее распространённых методов этого класса. В обзоре [32] показано, что MPC позволяет учитывать динамику робота, ограничения на управление и наличие препятствий. Алгоритм на каждом шаге решает задачу оптимизации на конечном горизонте прогнозирования, обеспечивая высокое качество траекторий. Основным недостатком является высокая вычислительная сложность.

Model Predictive Path Integral (MPPI) представляет собой стохастический вариант MPC, основанный на выборке большого числа возможных траекторий и оценке их стоимости [33, 34]. В работе [35] MPPI успешно применён для задачи агрессивного автономного вождения. Алгоритм опирается на локальную карту стоимостей (costmap), формируемую из априорной карты и данных сенсоров: в функцию стоимости каждой сэмплированной траектории напрямую включается штраф за столкновение с занятыми клетками costmap, что позволяет интегрировать глобальную и локальную информацию о среде. Преимуществами MPPI являются возможность работы с нелинейной динамикой и со сложными функциями стоимости. Недостатком является необходимость больших вычислительных ресурсов.

Алгоритм CCS-MPPI, предложенный в [36], объединяет MPPI с методом Covariance Steering и позволяет управлять не только средним значением состояния, но и его ковариацией, обеспечивая вероятностные гарантии соблюдения ограничений. Этот подход особенно эффективен в стохастических средах, но сложен в реализации.

Метод RMPPI (Robust MPPI), предложенный в [37], направлен на повышение робастности MPPI к ошибкам модели и внешним возмущениям. Он сочетает MPPI с трекинг-контроллером и механизмами безопасного распространения номинальной траектории.

Метод RA-MPPI (Risk-Aware MPPI), предложенный в [38], повышает робастность MPPI за счёт явного учёта хвостового риска на основе CVaR. Для каждой номинальной траектории разыгрывается серия возмущённых роллаутов, отражающих неопределённость исполнения управления. CVaR по этим роллаутам добавляется как штраф к стоимости траектории, смещая взвешивание Больцмана в сторону траекторий с низким хвостовым риском

без полного исключения рискованных сэмплов из рассмотрения.

Метод Adaptive MPP предложен в работе [39], в нём параметры модели и контроллера автоматически настраиваются под текущие условия среды. Это позволяет повысить устойчивость алгоритма при неточной модели динамики.

Оптимизационные методы обеспечивают наивысшее качество траекторий и наилучший учёт динамики робота и среды, однако их применение ограничено высокими вычислительными затратами и сложностью реализации.

2.3 Сравнительный анализ и выбор алгоритмов

Для систематизации полученных данных и выбора оптимального стека технологий был проведен сравнительный анализ алгоритмов. В таблицах 2.1, 2.2 представлены ключевые характеристики как глобальных, так и локальных планировщиков.

Для оценки эффективности используются следующие обозначения:

«+» — высокий показатель (высокая скорость работы, эффективная работа с динамикой, высокая точность);

«+-» — средний показатель (зависит от настроек, средние вычислительные затраты);

«-» — низкий показатель (медленная работа, неэффективность в динамике, риск локальных минимумов).

2.3.1 Обоснование выбора алгоритма D* Lite

Для решения задачи навигации в динамических средах в качестве глобального планировщика выбран алгоритм D* Lite.

В условиях непрерывного обновления карты сенсорными данными использование классического A* становится неэффективным: необходимость полной перестройки графа при каждом изменении среды создает критические вычислительные задержки. Алгоритм D* Lite нивелирует этот недостаток за счет механизма инкрементального поиска, при котором пересчитываются только участки пути, непосредственно затронутые изменениями. Это позволяет достичь оптимального баланса между глобальной эффективностью маршрута и высокой скоростью реакции, характерной для локальных планировщиков. Кроме того, относительная алгоритмическая простота D* Lite упрощает его интеграцию в гибридные навигационные системы и облегчает процесс отладочного тестирования.

2.4 Выводы по главе

Проведенный анализ подтверждает отсутствие универсального алгоритма, способного одновременно обеспечивать оптимальный глобальный охват и мгновенное избегание препятствий. Глобальные планировщики гарантируют достижение целевой точки и отсутствие тупиковых ситуаций, однако обладают высокой вычислительной инертностью. Напротив,

Таблица 2.1: Сравнение алгоритмов глобального планирования

Класс/ алгоритм	Работа в динамической среде	Вычислительная сложность	Основные преимущества	Основные недостатки
Графовые Dijkstra	—	+	Гарантированная оптимальность, простота	Полный пересчёт при изменениях
Графовые A*	—	+-	Быстрее Дейкстры, эвристический поиск	Зависит от эвристики
Графовые D*	+	+-	Инкрементальное перепланирование	Сложная реализация
Графовые D* Lite	+	+-	Эффективное обновление пути	Требует памяти
Графовые LPA*	+	+-	Повторное использование поиска	Медленнее D* Lite
Графовые ARA*	+-	+-	Быстрое первое решение	Временно субоптимален
Семплинг RRT	+-	—	Быстро находит путь	Плохое качество пути
Семплинг RRT*	+-	+	Оптимальные траектории	Медленная сходимости
Семплинг BIT*	+-	+-	Эвристическое ускорение	Сложная настройка
Семплинг RABIT*	+-	+-	Быстрая оптимизация	Зависит от локального оптимизатора
Семплинг Risk-DTRRT	+	+	Учет неопределённости	Сложность модели
Бионические GA	+-	+	Гибкость, адаптация	Медленная сходимости
Бионические ACO	+-	+-	Распределённый поиск	Чувствителен к параметрам
Бионические ABC	+-	+-	Простота реализации	Нестабильность
Бионические PSO	+-	+-	Быстро сходится	Локальные минимумы

локальные методы (такие как DWA или MPC) обеспечивают оперативную безопасность движения, но без глобального ориентира могут приводить к заикливаниям робота в сложных пространственных структурах.

Наиболее эффективным решением признана гибридная архитектура, в которой за инкрементальный пересчет маршрута отвечает алгоритм D^* Lite, а за безопасную отработку траектории в режиме реального времени — локальный планировщик. Такое сочетание минимизирует время простоя робота при изменении условий среды, обеспечивая баланс между глобальной оптимальностью пути и высокой скоростью реакции на динамические помехи.

Таблица 2.2: Сравнение алгоритмов локального планирования

Класс/ алгоритм	Работа в динамической среде	Вычислительная сложность	Основные преимущества	Основные недостатки
Классические контроллеры PID	—	—	Простота, высокая частота управления	Нет планирования, нет обхода препятствий
Классические контроллеры LQR	+-	+-	Оптимальность, устойчивость	Требуется линеаризации
Классические контроллеры PID+LQR	+-	+-	Повышенная устойчивость и точность	Сложность настройки
Геометрические DWA	+	—	Реактивность, практическая применимость	Локальные минимумы
Геометрические VO	+	+-	Учёт движущихся препятствий	Чувствительность к шуму
Геометрические APF	+-	—	Простота, высокая скорость	Локальные минимумы
Геометрические VFH	+	—	Эффективная работа с сенсорами	Нет глобальной оптимальности
Оптимизационные MPC	+	+	Учёт ограничений и динамики	Большие вычислительные затраты
Оптимизационные MPPI	+	+	Качественные траектории	Требуется параллелизма
Оптимизационные CCS-MPPI	+	+	Вероятностные гарантии безопасности	Сложная реализация
Оптимизационные RA-MPPI	+	+	Явный учёт хвостового риска	Дополнительные роллауты
Оптимизационные Adaptive MPPI	+	+	Адаптация к неопределённости	Сложность настройки

3 Экспериментальное сравнение алгоритмов A^* и D^* Lite

В данной главе проводится экспериментальное сравнение алгоритмов A^* и D^* Lite в статической и динамической средах. Цель экспериментов состоит в количественной оценке времени построения и перепланирования маршрута при изменении условий движения.

3.1 Общие концепции работы D^* Lite и поведение A^* в динамике

Алгоритм D^* Lite относится к инкрементальным методам поиска пути. Его ключевая идея заключается в повторном использовании результатов предыдущего поиска при частичных изменениях карты. Вместо полного пересчета всего маршрута алгоритм обновляет только те участки графа, для которых изменилась стоимость переходов или проходимость. За счет этого снижается вычислительная нагрузка в сценариях, где среда обновляется локально и часто.

```

procedure CalculateKey( $s$ )
{01"} return [ $\min(g(s), rhs(s)) + h(s_{start}, s) + k_m; \min(g(s), rhs(s))$ ];

procedure Initialize()
{02"}  $U = \emptyset$ ;
{03"}  $k_m = 0$ ;
{04"} for all  $s \in S$   $rhs(s) = g(s) = \infty$ ;
{05"}  $rhs(s_{goal}) = 0$ ;
{06"}  $U.Insert(s_{goal}, [h(s_{start}, s_{goal}); 0])$ ;

procedure UpdateVertex( $u$ )
{07"} if ( $g(u) \neq rhs(u)$  AND  $u \in U$ )  $U.Update(u, CalculateKey(u))$ ;
{08"} else if ( $g(u) \neq rhs(u)$  AND  $u \notin U$ )  $U.Insert(u, CalculateKey(u))$ ;
{09"} else if ( $g(u) = rhs(u)$  AND  $u \in U$ )  $U.Remove(u)$ ;

procedure ComputeShortestPath()
{10"} while ( $U.TopKey() < CalculateKey(s_{start})$  OR  $rhs(s_{start}) > g(s_{start})$ )
{11"}    $u = U.Top()$ ;
{12"}    $k_{old} = U.TopKey()$ ;
{13"}    $k_{new} = CalculateKey(u)$ ;
{14"}   if ( $k_{old} < k_{new}$ )
{15"}      $U.Update(u, k_{new})$ ;
{16"}   else if ( $g(u) > rhs(u)$ )
{17"}      $g(u) = rhs(u)$ ;
{18"}      $U.Remove(u)$ ;
{19"}     for all  $s \in Pred(u)$ 
{20"}       if ( $s \neq s_{goal}$ )  $rhs(s) = \min(rhs(s), c(s, u) + g(u))$ ;
{21"}       UpdateVertex( $s$ );
{22"}   else
{23"}      $g_{old} = g(u)$ ;
{24"}      $g(u) = \infty$ ;
{25"}     for all  $s \in Pred(u) \cup \{u\}$ 
{26"}       if ( $rhs(s) = c(s, u) + g_{old}$ )
{27"}         if ( $s \neq s_{goal}$ )  $rhs(s) = \min_{s' \in Succ(s)} (c(s, s') + g(s'))$ ;
{28"}       UpdateVertex( $s$ );

procedure Main()
{29"}  $s_{last} = s_{start}$ ;
{30"} Initialize();
{31"} ComputeShortestPath();
{32"} while ( $s_{start} \neq s_{goal}$ )
{33"}   /* if ( $g(s_{start}) = \infty$ ) then there is no known path */
{34"}    $s_{start} = \arg \min_{s' \in Succ(s_{start})} (c(s_{start}, s') + g(s'))$ ;
{35"}   Move to  $s_{start}$ ;
{36"}   Scan graph for changed edge costs;
{37"}   if any edge costs changed
{38"}      $k_m = k_m + h(s_{last}, s_{start})$ ;
{39"}      $s_{last} = s_{start}$ ;
{40"}     for all directed edges ( $u, v$ ) with changed edge costs
{41"}        $c_{old} = c(u, v)$ ;
{42"}       Update the edge cost  $c(u, v)$ ;
{43"}       if ( $c_{old} > c(u, v)$ )
{44"}         if ( $u \neq s_{goal}$ )  $rhs(u) = \min(rhs(u), c(u, v) + g(v))$ ;
{45"}         else if ( $rhs(u) = c_{old} + g(v)$ )
{46"}           if ( $u \neq s_{goal}$ )  $rhs(u) = \min_{s' \in Succ(u)} (c(u, s') + g(s'))$ ;
{47"}       UpdateVertex( $u$ );
{48"}     ComputeShortestPath();

```

Рис. 3.1: Иллюстрация принципа инкрементального перепланирования в D^* Lite. Источник [7].

В отличие от инкрементальных методов, классический A^* в динамической среде функционирует как последовательность независимых запусков. После каждого шага, в котором накапливаются изменения карты, алгоритм заново строит маршрут от текущей позиции до цели, не используя результаты предыдущих вычислений. В итоге при увеличении числа шагов и частоты изменений суммарные вычислительные затраты быстро возрастают.

3.2 Модель и условия эксперимента

Целью исследования было сравнение эффективности классического и инкрементального подходов в различных условиях навигации.

В качестве карты использовался стандартный бенчмарк «Arena» (с ресурса MovingAI) — открытое пространство размером примерно 49×49 клеток с блоками статических препятствий и распределёнными группами динамических объектов.

Параметр сложности конфигурации (n_dst) определялся как минимальное манхэттенское расстояние между точками старта и финиша: $d_M = |x_{start} - x_{goal}| + |y_{start} - y_{goal}| \geq n$. Для каждой итерации (от 0_dst до 70_dst) генерировалось 1000 случайных пар точек, что обеспечило высокую статистическую достоверность.

В ходе тестов от 1 до 15 препятствий за каждый ход совершали случайное перемещение на одну клетку (вверх, вниз, влево или вправо). Для алгоритма A^* проверялись две стратегии пересчёта: в режиме *step* путь пересчитывается один раз в конце хода после того, как все препятствия совершили движение; в режиме *change* путь пересчитывается мгновенно после каждого единичного перемещения любого препятствия.

3.3 Результаты экспериментов

Измерения для статического режима представлены на рисунке 3.2. По оси абсцисс расположены пороги на минимальную дистанцию между стартом и целью (0, 10, 30, 50, 70 клеток), а по оси ординат — усредненные времена построения маршрута в миллисекундах.

Измерения для динамического режима представлены на рисунке 3.3. График отражает зависимость времени вычислений от числа препятствий, изменяющих положение за один ход (от 1 до 15), а также демонстрирует различие между режимами *step* и *change* для алгоритма A^* .

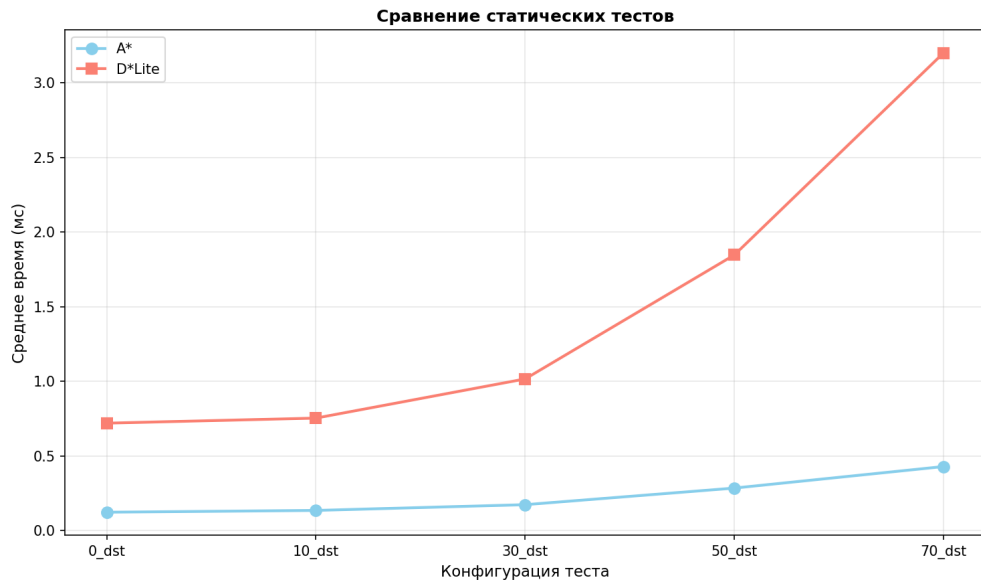


Рис. 3.2: Сравнение времени построения маршрута в статической среде.

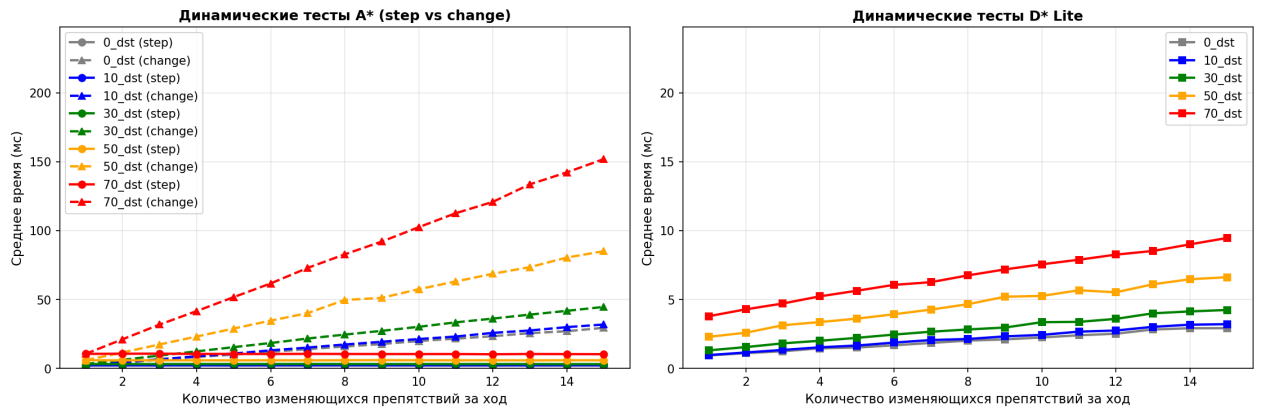


Рис. 3.3: Сравнение времени работы в динамической среде при различном числе подвижных препятствий.

3.4 Анализ производительности в статической среде

В условиях неизменной карты алгоритм A^* показал абсолютное преимущество (рис. 3.2). Его вычислительная сложность при разовом поиске значительно ниже, чем у конкурента.

Время выполнения A^* растёт крайне медленно. Даже при максимальном удалении точек (70_dst) среднее время поиска составляет всего 0,42 мс. Это подтверждает, что для однократного планирования на статичной карте A^* является оптимальным решением.

D^* Lite демонстрирует более тяжеловесный старт — до 3,2 мс на аналогичной дистанции. Это связано с накладными расходами на инициализацию сложных структур данных и приоритетных очередей, которые необходимы алгоритму для будущей адаптации к изменениям, но в статике являются избыточными.

3.5 Анализ производительности в динамической среде

При введении движущихся препятствий ситуация кардинально меняется (рис. 3.3). Ключевым фактором становится способность алгоритма переиспользовать предыдущие вычисления.

Алгоритм A^* не обладает «памятью» о предыдущих итерациях, что приводит к полной перепланировке при каждом изменении карты. В режиме *change* при 15 изменениях за ход время пересчёта на дистанции 70_dst взлетает до 150...200 мс, поскольку алгоритм выполняет 15 полноценных поисков за один цикл. Режим *step* снижает нагрузку до 1...15 мс (в среднем около 8 мс), однако даже этот показатель заметно уступает D^* Lite при высокой плотности изменений.

Алгоритм D^* Lite специально разработан для работы в меняющемся окружении. Его время обработки изменений остаётся стабильным в диапазоне 3...10 мс независимо от интенсивности движения препятствий. На карте «Arena» это проявляется наиболее ярко: поскольку объекты перемещаются всего на одну клетку, D^* Lite обновляет лишь локальные веса в своём графе, не затрагивая те части пути, которые остались актуальными. Это делает его в 15...20 раз быстрее A^* в режиме *change* при высокой плотности динамики.

Наблюдаемые результаты подтверждают принципиальное различие вычислительных стратегий алгоритмов. В статической среде A^* демонстрирует явное превосходство благодаря элегантной архитектуре поиска в графе. Однако в динамической среде при росте числа препятствий и частоте обновлений преимущество полностью переходит к D^* Lite за счет инкрементального характера обновлений. Разрыв в производительности становится катастрофичным для A^* в режиме *change*: при 15 движущихся объектах производительность падает по крайней мере в 15...20 раз по сравнению с D^* Lite.

3.6 Выводы по главе

Проведенное сравнение показало, что выбор алгоритма существенно зависит от характера среды и требований приложения.

Таблица 3.1: Сравнительные характеристики A^* и D^* Lite

Параметр сравнения	A^* (A-Star)	D^* Lite
Статическая скорость (мин–макс, среднее)	0,12–0,42 мс (~0,27 мс)	0,7–3,2 мс (~1,9 мс)
Динамика: режим <i>step</i> (мин–макс, среднее)	1,0–15,0 мс (~8,0 мс)	3,0–10,0 мс (~6,5 мс)
Динамика: режим <i>change</i> (мин–макс, среднее)	1,0–200,0 мс (~100,0 мс)	3,0–10,0 мс (~6,5 мс)

Для разового построения пути в статических условиях классический A^* эффективен благодаря простоте и низким накладным расходам. Однако для задач с регулярными изменениями карты, особенно при интенсивной динамике, значительно более устойчивым по времени выполнения оказывается D^* Lite.

Таким образом, эксперимент подтверждает, что для задач разовой навигации предпочтителен A^* благодаря его вычислительной легкости. Однако в условиях интенсивной ди-

намики и необходимости мгновенной реакции на изменения среды (режим *change*) D^* Lite является единственным масштабируемым решением, гарантирующим предсказуемое время отклика.

4 Экспериментальное сравнение локальных планировщиков MPPI и RA-MPPI

В данной главе проводится экспериментальное сравнение локальных планировщиков MPPI и RA-MPPI в статической и динамической средах. Цель экспериментов состоит в количественной оценке качества траекторий и надёжности навигации при различных уровнях динамики окружения.

4.1 Общие концепции работы MPPI и RA-MPPI

MPPI (Model Predictive Path Integral) — стохастический локальный планировщик, основанный на одновременной оценке большого числа возможных траекторий [33, 35]. На каждом шаге алгоритм генерирует K возмущённых траекторий из текущей позиции робота: к номинальному управлению $\bar{u}$ (в данной реализации — единичному вектору, направленному к цели; в общем случае MPPI использует предыдущую оптимизированную последовательность управлений) прибавляется гауссовый шум $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$, после чего каждая траектория разворачивается на горизонте T шагов и оценивается скалярной функцией стоимости

$$J_k = \sum_{t=1}^T \left[c_{\text{obst}} \cdot \mathbb{I}[s_t^{(k)} \in \mathcal{O}] + w_s \cdot d(s_t^{(k)}, g) \right] + w_T \cdot d(s_T^{(k)}, g),$$

где $s_t^{(k)}$ — состояние k -й траектории на шаге t , $\mathcal{O}$ — множество клеток-препятствий, $\mathbb{I}[\cdot]$ — скобка Айверсона (равна 1, если условие выполнено, и 0 иначе), $d(s, g)$ — евклидово расстояние от состояния s до цели g , w_s и w_T — весовые коэффициенты промежуточного и терминального слагаемых, c_{obst} — штраф за столкновение с препятствием. Терминальное слагаемое $w_T \cdot d(s_T^{(k)}, g)$ суммируется поверх последнего слагаемого ряда и тем самым усиливает значимость финальной точки траектории.

Управление обновляется как взвешенная сумма возмущений с весами Больцмана (softmin по стоимости), в которых из аргумента экспоненты вычитается $J_{\min} = \min_j J_j$ для численной устойчивости:

$$w_k = \frac{\exp\left(-\frac{J_k - J_{\min}}{\lambda}\right)}{\sum_{j=1}^K \exp\left(-\frac{J_j - J_{\min}}{\lambda}\right)}, \quad \sum_{k=1}^K w_k = 1.$$

Параметр $\lambda > 0$ задаёт «температуру» распределения: при малых λ управление определяется преимущественно наилучшей траекторией, при больших — усредняется по всему ансамблю.

RA-MPPI (Risk-Aware MPPI) расширяет MPPI явным учётом хвостового риска на основе условного значения под риском (CVaR) [38]. Для каждой из K номинальных траекторий разыгрывается N_d возмущённых роллаутов с дополнительным случайным шумом $\delta \sim \mathcal{N}(0, \sigma_d^2 I)$, имитирующим неопределённость исполнения управления. По полученным N_d

стоимостям вычисляется CVaR_α — среднее по худшим $\lceil (1 - \alpha)N_d \rceil$ значениям:

$$\hat{J}_k = J_k + \gamma \cdot \text{CVaR}_\alpha(\{J_k^{(n)}\}_{n=1}^{N_d}),$$

где J_k — стоимость номинального роллаута, $J_k^{(n)}$ — стоимость n -го возмущённого роллаута, $\gamma > 0$ — вес штрафа за риск. Штраф добавляется только тогда, когда CVaR превышает заданный порог C_u ; иначе $\hat{J}_k = J_k$. Затем стандартная схема взвешивания Больцмана применяется к скорректированным стоимостям $\hat{J}_k$, и RA-MPPI сводится к классическому MPPI, если CVaR ни для одной траектории не нарушает порог.

4.2 Модель и условия эксперимента

Эксперименты выполнялись на том же бенчмарке «Арена», что и сравнение глобальных планировщиков: карта размером 49×49 клеток со статическими препятствиями и группами динамических объектов.

Оба планировщика использовали $K = 50$ сэмплов при горизонте $T = 12$ шагов, температурный параметр $\lambda = 1,5$, стандартное отклонение шума $\sigma = 0,8$, штраф за столкновение $c_{\text{obst}} = 200$, $w_s = 0,1$, $w_T = 15,0$. Для RA-MPPI дополнительно задавались уровень доверия $\text{CVaR } \alpha = 0,8$, число возмущённых роллаутов $N_d = 20$, стандартное отклонение возмущений $\sigma_d = 0,5$, вес штрафа за риск $\gamma = 1,0$ и порог активации штрафа $C_u = 0$.

Из полного набора задач отбирались 100 случайных пар «старт — цель»; ограничение на число шагов составляло 500 в статическом и 50 000 в динамическом режиме. В отличие от тестов глобальных планировщиков, количество задач было ограничено 100 ввиду большей вычислительной стоимости сэмплирующих алгоритмов на каждом шаге навигации. Аналогично предыдущим тестам, препятствия за каждый ход совершали случайное перемещение на одну клетку; тестировались три уровня загрузки: 1, 5 и 10 подвижных препятствий.

В отличие от глобальных планировщиков, для которых основным критерием являлось время перепланирования, оценка локальных методов строилась на четырёх показателях: *успешность* — доля задач, для которых найден путь до цели; *время планирования* (мс) — суммарное стеновое время построения пути; *извилистость* — отношение длины пути к прямому расстоянию до цели (1,0 соответствует прямолинейному маршруту); *доля опасных сближений* — доля шагов, на которых хотя бы один из ортогональных соседей являлся препятствием.

4.3 Результаты экспериментов

Измерения для статического режима представлены на рисунке 4.1. На графике сопоставляются время планирования и число шагов обоих алгоритмов по всем 100 тестовым задачам.

Измерения для динамического режима представлены на рисунке 4.2. График отражает зависимость ключевых показателей качества навигации от числа подвижных препятствий

(1, 5 или 10).

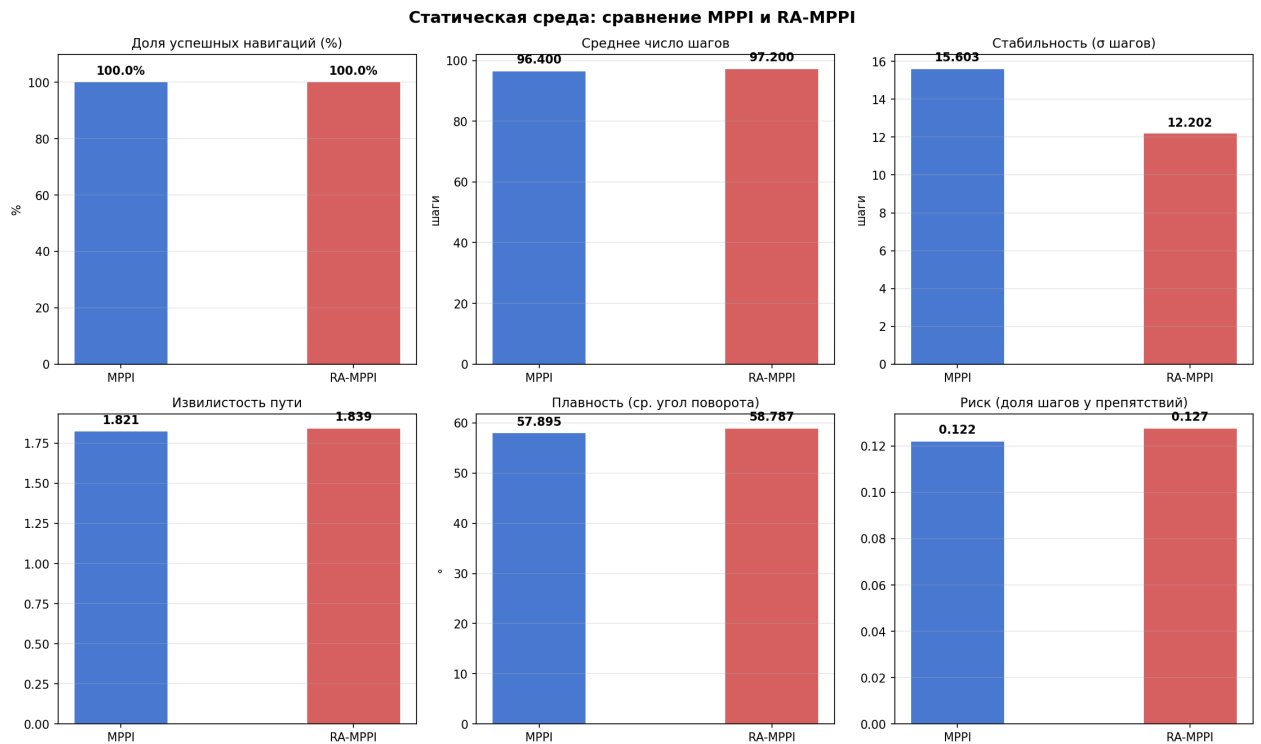


Рис. 4.1: Сравнение показателей MPPI и RA-MPPI в статической среде.

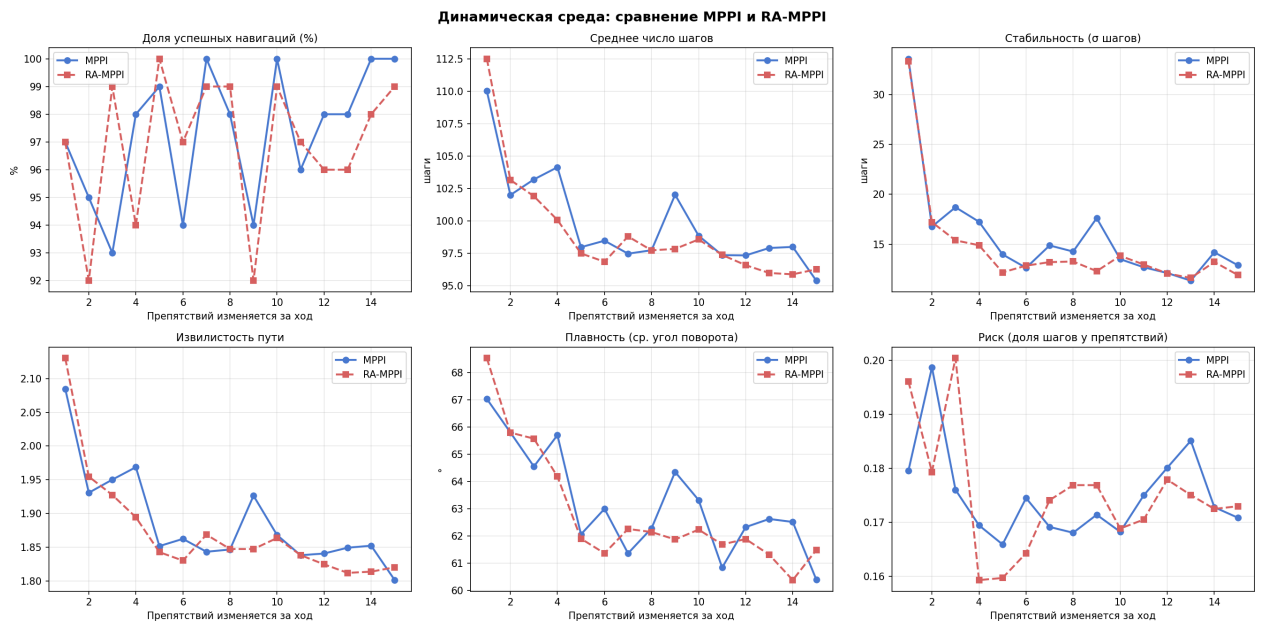


Рис. 4.2: Сравнение показателей MPPI и RA-MPPI в динамической среде при различном числе подвижных препятствий.

4.4 Анализ производительности в статической среде

В условиях неизменной карты оба алгоритма показали абсолютную успешность (рис. 4.1): все 100 задач решены без единого отказа как для MPPI, так и для RA-MPPI.

Для МРРІ среднее время построения маршрута составило 1,7 мс, среднее число шагов — 96,4, извилистость — 1,821, доля опасных ситуаций — 12,2%. RA-MРРІ показал сопоставимое время — 1,7 мс — и незначительно большее число шагов (97,2 шага, извилистость 1,839), доля опасных ситуаций составила 12,7%. Небольшое увеличение числа опасных сближений объясняется тем, что механизм CVaR-штрафа является мягким: он не исключает рискованные сэмплы полностью, а лишь увеличивает их стоимость, что при низкой частоте препятствий в статической среде не приводит к существенному изменению выбираемых траекторий.

В целом оба метода демонстрируют сопоставимую эффективность в статической среде: МРРІ незначительно менее консервативен, а RA-MРРІ незначительно увеличивает длину пути.

4.5 Анализ производительности в динамической среде

Введение движущихся препятствий выявило нетривиальную зависимость (рис. 4.2): при малом числе объектов (1 за ход) оба алгоритма демонстрировали сниженную успешность, тогда как при большем числе препятствий (5 и более) надёжность навигации возрастала.

При малом числе объектов (1 за ход) МРРІ и RA-MРРІ показали схожую успешность — 97 из 100 задач для обоих алгоритмов; среднее число шагов возрастало до 110,0 и 112,5 соответственно, а извилистость значительно увеличивалась (2,085 и 2,130). Снижение успешности объясняется поведением сэмплирующего планировщика в условиях узких проходов: единственное движущееся препятствие может блокировать все ортогональные соседи робота одновременно, лишая алгоритм любого допустимого направления движения.

При 5 подвижных объектах МРРІ достигал 99% успешности, RA-MРРІ — 100%; среднее число шагов снижалось до 98,0 и 97,5 соответственно, а извилистость приближалась к статическому уровню (1,852 и 1,843). При 10 подвижных объектах МРРІ достигал 100% успешности, RA-MРРІ — 99% (различие в 1 случай из 100, в пределах статистической погрешности на данной выборке); числа шагов и извилистость практически совпадали (98,9 и 98,6 шага).

Наблюдаемые результаты подтверждают, что МРРІ и RA-MРРІ занимают близкие точки компромисса между надёжностью и консервативностью. Оба алгоритма ведут себя эквивалентно во всех исследованных режимах в пределах статистической погрешности на выборке из 100 задач.

4.6 Выводы по главе

Проведённое сравнение показало, что МРРІ и RA-MРРІ демонстрируют принципиально схожие вычислительные характеристики, однако различаются по устойчивости и качеству траекторий в зависимости от уровня динамики среды.

Таблица 4.1: Сравнительные характеристики МРРІ и RA-МРРІ

Параметр сравнения	МРРІ	RA-МРРІ
Статика: успешность	100%	100%
Статика: среднее время, мс	1,7	1,7
Статика: среднее число шагов	96,4	97,2
Статика: извилистость	1,821	1,839
Статика: доля опасных ситуаций	12,2%	12,7%
Динамика (1 препятствие): успешность	97%	97%
Динамика (1 препятствие): среднее число шагов	110,0	112,5
Динамика (1 препятствие): извилистость	2,085	2,130
Динамика (5 препятствий): успешность	99%	100%
Динамика (5 препятствий): среднее число шагов	98,0	97,5
Динамика (10 препятствий): успешность	100%	99%
Динамика (10 препятствий): среднее число шагов	98,9	98,6

МРРІ и RA-МРРІ демонстрируют практически идентичное время планирования (1,7 мс), а по успешности различаются не более чем на 1 процентный пункт во всех тестовых режимах — в пределах статистической погрешности на 100 задачах. RA-МРРІ не уступает МРРІ по успешности в режиме малой динамики (97% против 97%), тогда как задержка, вносимая возмущёнными роллаутами, компенсируется параллельным вычислением в едином проходе оценки сэмплов.

Таким образом, оба алгоритма пригодны для задач локальной навигации и демонстрируют практически одинаковую успешность во всём диапазоне исследованных условий. RA-МРРІ предпочтителен там, где необходима явная мера хвостового риска и её интерпретируемость; МРРІ проще в настройке при фиксированной статической среде. На экстремальных режимах оба метода эквивалентны в пределах статистической погрешности.

5 Реализация навигационного стека в ROS 2 и сквозное тестирование

В предыдущих главах исследование выполнялось в виде самостоятельных программных моделей, фокусирующихся на отдельных метриках выбранных алгоритмов: скорости перепланирования для A^*/D^* Lite и качестве траекторий для MPPI/RA-MPPI. Цель настоящей главы — интегрировать эти алгоритмы в единый навигационный стек и подтвердить их совместную работоспособность на физическом симуляторе мобильного робота.

5.1 Архитектура и используемые компоненты

Реализация выполнена на стеке ROS 2 Jazzy с физическим симулятором Gazebo Sim Harmonic (gz-sim) и фреймворком навигации Nav2. В качестве робота выбран TurtleBot3 Burger — двухколёсная дифференциальная платформа радиусом 10 см с лазерным сканером 360°, что соответствует типичной конфигурации мобильных роботов в логистике.

Архитектура построена на модели «глобальный планировщик + локальный контроллер», разделение которой обосновано в главе 2. Глобальный уровень получает на вход карту занятости и текущее положение робота, выдаёт опорный путь до цели; локальный уровень преобразует опорный путь в управляющие команды по скорости `cmd_vel`, реагируя на показания сенсоров и движущиеся объекты.

Все четыре исследуемых конфигурации строятся комбинированием двух глобальных планировщиков — A^* (NavfnPlanner с `use_astar=true`) и D^* Lite (собственный плагин `dstar_lite_planner`) — с двумя локальными контроллерами: *MPPI* (стандартный `nav2_mppi_controller`) и *RA-MPPI* (собственный плагин `ramppi_controller`).

Соответствие пар фиксируется отдельным файлом параметров Nav2 (вида `nav2_params/my_nav2_params_<planner>_<controller>.yaml`), общая структура которого включает `planner_server`, `controller_server`, `behavior_server`, `bt_navigator`, два слоя `costmap` (глобальный и локальный) и стандартный набор узлов восстановления (`spin`, `backup`, `wait`).

5.2 Разработанные программные пакеты

В составе работы реализованы два собственных пакета ROS 2 и набор вспомогательных скриптов для построения экспериментов и анализа bag-файлов.

5.2.1 Реализация D^* Lite в виде плагина Nav2

Пакет реализует D^* Lite как `nav2_core::GlobalPlanner`-плагин. Реализация включает четыре исходных файла: `dstar_lite.cpp` (ядро алгоритма: *rhs*-обновление, очередь с приоритетом по ключу $[k_1, k_2]$, перерасчёт изменённых вершин), `field.cpp` (преобразование `nav2_costmap_2d::Costmap2D` в граф 4-связности с учётом порога летальной стоимости), `heuristic_functions.cpp` (манхэттенская эвристика $h(s) = |s_x - g_x| + |s_y - g_y|$) и

`dstar_lite_planner.cpp` (адаптер плагина: жизненный цикл, извлечение текущего `costmap` через `costmap_ros_->getCostmap()`, конвертация результата в `nav_msgs::msg::Path`). Внутрь ядра *D* Lite* сохранены все инкрементальные свойства, ранее подтверждённые в главе 3. В рамках Nav2-плагина каждый вызов `createPlan()` строит `Field` и `DstarLite` от текущего состояния `costmap` и вычисляет полный путь: интерфейс `nav2_core::GlobalPlanner` не предоставляет планировщику информации о том, какие именно ячейки `costmap` изменились между вызовами, а перепланирование инициируется самим Nav2 по таймеру `expected_planner_frequency`. В рассматриваемых сценариях это не сказывается на качестве траекторий: карта компактна (112×103 ячеек), полный поиск занимает единицы миллисекунд и сопоставим по времени с накладными расходами Nav2 на формирование результата.

5.2.2 Реализация RA-MPPI в виде плагина Nav2

Пакет реализует RA-MPPI как `nav2_core::Controller`-плагин. Ядро (`mppi.cpp`, `ramppi.cpp`) воспроизводит реализацию модельного эксперимента из главы 4: K номинальных траекторий, оценка функции стоимости, для каждой траектории — N_d возмущённых роллаутов с дополнительным шумом σ_d , вычисление $CVaR_\alpha$ и добавление штрафа $\gamma \cdot CVaR$, затем стандартное взвешивание Больцмана по скорректированным стоимостям. Адаптер (`ramppi_controller.cpp`) реализует интерфейсный слой ROS 2: подписка на глобальный путь, преобразование TF-фреймов между `map` и `odom/costmap`, вычисление точки преследования (pure-pursuit lookahead), формирование `geometry_msgs::msg::TwistStamped` и проверка достижения цели. Класс `Field` (`field.cpp`) кэширует локальный `costmap` для быстрой проверки столкновений сэмплируемых траекторий.

5.2.3 Скрипты построения и анализа экспериментов

Эксперимент опирается на набор `bash`- и `Python`-скриптов в директории `scripts/`. `move_cylinders.cpp` — вспомогательная программа на C++, которая через `gz-transport` вызывает сервис `/world/default/set_pose` и кинематически перемещает каждый цилиндр по синусоиде с частотой 10 Гц; использование C++ вместо повторного вызова `gz service` из `bash` продиктовано измерением: каждый `subprocess`-вызов добавляет ~ 280 мс задержки, что не позволяет достичь требуемой частоты при числе цилиндров более двух. `build_move_cylinders.sh` собирает данную программу против установленных библиотек `gz-transport` и протобуферов; исполняемый файл помещается в `build/move_cylinders`.

`run_one_test.sh` запускает один тест: чистит оставшиеся процессы и DDS-кэш, стартует Gazebo и процесс `move_cylinders` для динамических миров, поднимает Nav2, ожидает `/odom` и AMCL-локализации (цикл публикации `/initialpose` с проверкой `/amcl_pose`, до 60 с), затем ожидает action-сервера `/navigate_to_pose` и передаёт управление `record_test.sh`. `run_batch.sh` последовательно прогоняет все 16 сценариев (4 пары планировщик-контроллер $\times$ 4 мира): каждый сценарий запускается ровно 10 раз с полной перезагрузкой Gazebo, Nav2 и процесса `move_cylinders` между прогонами, результаты усредняются по успешным.

`record_test.sh` отправляет цель через action-сервер `/navigate_to_pose` и параллельно записывает топики `/odom`, `/amcl_pose`, `/plan`, `/collision_monitor_state` в bag-файл формата `msar`. `analyze_bag.py` выполняет пост-обработку: десериализует bag, вычисляет длину пути по `/odom`, оценивает успешность по положению робота в map-фрейме и подсчитывает опасные сближения из `/collision_monitor_state`.

5.3 Среда тестирования и сценарии

Базовая карта — стандартный мир `turtlebot3_world`, представляющий собой квадрат $4,2 \times 4,0$ м, ограниченный стенами, с регулярной сеткой 3×3 цилиндрических колонн радиусом 15 см. Все эксперименты используют общий маршрут: старт $(-2,0, -0,5)$, цель $(1,8, 0,5)$, что соответствует диагональному пересечению карты с прохождением через верхний коридор (между северным рядом колонн и стеной). Базовая карта показана на рисунке 5.1: зелёный круг — точка старта, красный — цель.

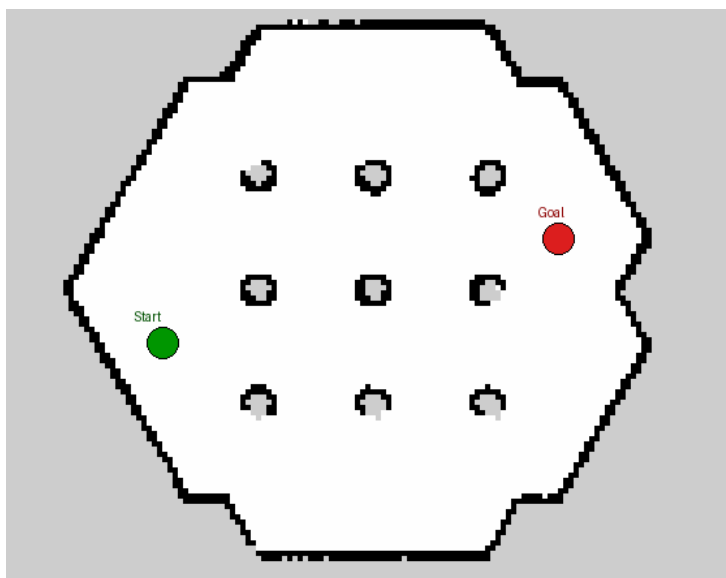


Рис. 5.1: Статическая карта `turtlebot3_world`: 3×3 колонн, старт $(-2,0; -0,5)$, цель $(1,8; 0,5)$.

Для оценки устойчивости к движущимся препятствиям сгенерированы три динамических мира: `dynamic_1` (одно препятствие, целенаправленно пересекающее траекторию), `dynamic_5` (умеренная плотность, 5 препятствий), `dynamic_10` (высокая плотность, 10 препятствий). Каждое препятствие движется кинематически по синусоиде вдоль своего коридора. В сценариях `dynamic_1` и `dynamic_5` все цилиндры используют единые параметры (период 17 с, амплитуда 0,55 м), что обеспечивает пиковую скорость 0,20 м/с. В сценарии `dynamic_10` периоды варьируются в диапазоне 12–20 с, амплитуды — в диапазоне 0,35–0,55 м, что даёт пиковые скорости 0,13–0,21 м/с и моделирует более разнородную динамическую среду. Указанный диапазон скоростей сопоставим с целевой скоростью робота ($\sim 0,25$ м/с) и позволяет локальному контроллеру в принципе предсказать траекторию препятствия на горизонте планирования. Расположение цилиндров и их коридоры движения для трёх плотностей по-

казаны на рисунке 5.2: синие круги — начальные позиции, светло-синие линии — допустимые отрезки синусоидального движения.

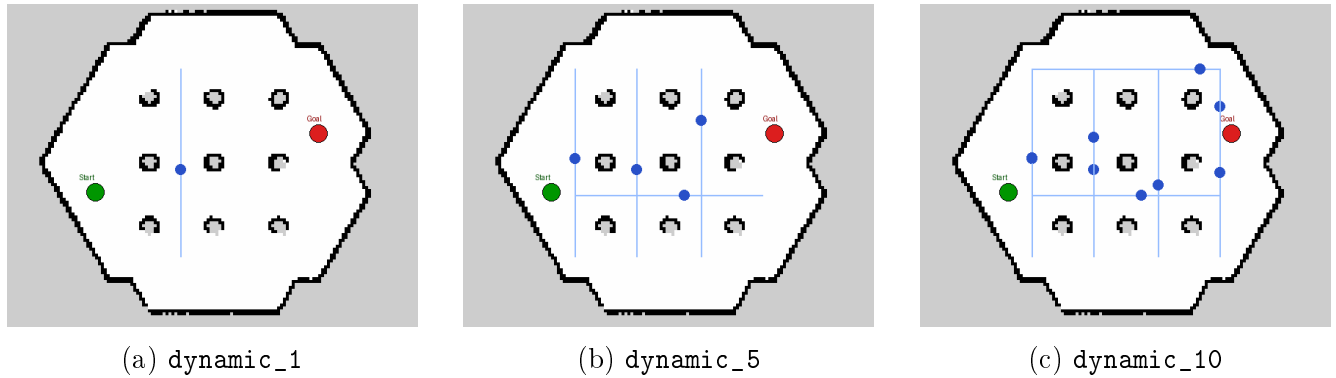


Рис. 5.2: Карты с подвижными цилиндрическими препятствиями.

На каждом сценарии фиксируются пять метрик: *успешность* — достиг ли робот окрестности цели (толерантность 0,35 м) в пределах 120-секундного бюджета; *итоговое отклонение* (`dist_to_goal`) — расстояние от финальной точки робота до цели; *длина пути* — суммарное смещение по одометрии; *длительность* — время от первого до последнего сообщения `/amcl_pose`; *опасные сближения* — число эпизодов срабатывания `collision_monitor`, разделённых паузой более 2 с.

5.4 Возникшие проблемы и принятые решения

Сборка и запуск интегрированного стека потребовали решения ряда практических проблем, не описанных непосредственно в литературе по алгоритмам. Ниже перечислены наиболее существенные из них.

5.4.1 Проблемы интеграции Nav2

Несовместимость типов сообщений. После обновления Nav2 в Jazzy ряд узлов перешёл с `geometry_msgs::Twist` на `TwistStamped`; одновременная работа узлов разных версий вызывала тихий разрыв связи между `controller_server` и `behavior_server`. Потребовалась явная унификация типа сообщений в пяти конфигурационных узлах Nav2.

Кэш DDS shared memory. Перезапуск Gazebo между тестами оставлял в `/dev/shm/` файлы вида `fastrtps_*`, в результате чего при следующем запуске возникала ошибка «invalid allocator». С другой стороны, очистка кэша между мирами разрушала и `discovery`-кэш демона ROS 2, что приводило к невидимости мостовых топиков на 30+ секунд. Принятое решение — однократная очистка `/dev/shm/fastrtps_*` перед стартом первого Gazebo, без последующих очисток.

Готовность action-сервера `/navigate_to_pose`. Отправка цели сразу после `ros2 launch nav2_bringup` приводила к её потере: `bt_navigator` ещё не успевал зарегистрировать сервер. Решение — цикл активного опроса, ожидающий появления action-сервера в выводе `ros2 action list` перед каждым тестом (тайм-аут 90 с).

Локализация AMCL. Однократная публикация `/initialpose` нередко терялась — AMCL не успевал подписаться. Решение — цикл с дедлайном 60 с: на каждой итерации повторно публикуется `/initialpose` и проверяется появление сообщения в топике `/amcl_pose`; при успешном подтверждении локализации цикл завершается. Первый запуск также требует предварительного ожидания топика `/odom`, поскольку TF-буфер до этого момента пуст.

5.4.2 Адаптация алгоритмических компонентов

Связность графа D* Lite. Реализация использует 4-связность графа и манхэттенскую эвристику $h(s) = |s_x - g_x| + |s_y - g_y|$. Такой выбор соответствует требованию допустимости эвристики при равных весах рёбер и существенно упрощает инкрементальное обновление множества изменённых вершин: при модификации `costmap` пересчёту подлежит ограниченное число соседей. Возникающие на диагональных участках лестничные траектории сглаживаются локальным контроллером в рамках штрафа `PathAlignCritic` и `pure-pursuit`-цели без существенного замедления движения.

Pure-pursuit lookahead в RA-MPPI. Прямое использование первой точки глобального плана как промежуточной цели приводило к колебаниям контроллера у криволинейных участков пути. Применён классический `pure-pursuit`: ищется ближайшая точка плана на расстоянии $\geq L_{\text{lookahead}}$ от робота, она и используется в качестве цели MPPI. Значение $L_{\text{lookahead}} = 0,80$ м даёт компромисс между предсказательной способностью и агрессивностью отклонений от глобального плана.

Параметры пучка траекторий σ и K . Значения, унаследованные из главы 4 ($K = 50$, $\sigma = 0,8$), в условиях полной системы Nav2 оказались недостаточными: зашумление `costmap` препятствиями динамической природы и задержки TF-преобразований приводили к преждевременной концентрации весов Больцмана на узкой группе сэмплов и потере устойчивости управления. В ROS-реализации число сэмплов увеличено до $K = 150$, а стандартное отклонение шума управления — до $\sigma = 1,0$, что обеспечивает более широкий пучок при одновременном росте статистической надёжности оценки средней стоимости.

SimpleProgressChecker. Стандартный `progress checker` считает робот «застрявшим», если он не сместился на $\geq 0,5$ м за 10 с. В плотных динамических сценариях робот может в течение нескольких секунд ожидать прохождения препятствия, что вызывало ложные срабатывания и переход в режим восстановления. Бюджет был смягчён до 0,05 м за 20 с.

5.4.3 Физика динамических препятствий

Наибольших усилий при разработке потребовала реализация подвижных препятствий. Попытки физически-обусловленного движения (`TrajectoryFollower` с `waypoints`, `line-mode`, призматический `joint` к миру) приводили к застреванию цилиндров на колоннах карты или к дрейфу на десятки сантиметров за тест. В качестве основного решения выбран кинематический телепорт через сервис `/world/default/set_pose`, выполняемый на 10 Гц отдельным процессом `move_cylinders`.

В рамках данного подхода потребовалось решить задачу предотвращения смещения препятствия при контакте с корпусом робота: при использовании `<static>true</static>` в SDF и одновременном вызове `set_pose` цилиндр в редких случаях терял жёсткость и сдвигался под действием робота. Решение — комбинация трёх атрибутов: `<static>true</static>` у модели, `<kinematic>true</kinematic>` у link и тяжёлый `<inertial>` (1000 кг) с отключённой гравитацией; такая избыточная конфигурация удерживает цилиндр неподвижным даже если одна из этих опций будет проигнорирована конкретной версией gz-sim.

Симметричная задача возникла в связи с обратным явлением — проникновением препятствия сквозь корпус робота: запланированная синусоидальная позиция могла оказаться внутри его габаритов. Введена зона безопасности 0,30 м (сумма радиусов плюс малый запас): при попадании запланированной позиции цилиндра в эту зону цилиндр удерживается в текущей координате и не смещается в сторону робота. Такое поведение принципиально отличается от уклонения цилиндра: оно сохраняет реалистичность твёрдого неподвижного препятствия и требует от локального контроллера активного объезда.

Дополнительно потребовалось исключить возможность взаимной блокировки, при которой робот ожидал бы освобождения прохода, а препятствие — ухода робота. С этой целью на время удержания цилиндра в зоне безопасности параллельно замораживается его эффективное время t_{eff} : фаза синусоиды не продвигается, пока сохраняется перекрытие. После выхода робота из зоны безопасности движение продолжается с той же фазы, не порождая скачкообразного смещения цилиндра. Дополнительно шаг изменения положения за один тик ограничен величиной 0,10 м, что предотвращает резкие рывки при последующем восстановлении движения.

5.5 Линейное предсказание траекторий динамических препятствий

Стандартный контроллер `nav2_mppi_controller` не различает статические и динамические препятствия: стоимость роллаута определяется по текущей карте стоимостей, которая обновляется лишь на следующем цикле Nav2. Для конфигураций с собственным контроллером `RAMPPIController` (пары A^* +RA-MPPI и D^* Lite+RA-MPPI) реализовано линейное предсказание положений цилиндров непосредственно внутри оценщика стоимости роллаутов. Конфигурации со стандартным `nav2_mppi_controller` предсказания не выполняют и воспринимают движущиеся цилиндры как статические объекты, сместившиеся к текущему обновлению costmap.

Механизм IPC. Процесс `move_cylinders` на каждом тике (10 Гц) записывает текущее положение и аналитическую скорость каждого цилиндра в файл `/tmp/cylinder_states` (атомарная замена через временный файл):

```
cylinder_0  x  y  vx  vy
cylinder_1  x  y  vx  vy
...
```

Скорость вычисляется аналитически как производная синусоиды: $v = A \cdot \frac{2\pi}{T} \cos\left(\frac{2\pi}{T}t + \varphi\right)$.

Интеграция в RAMPPIController. Контроллер `RAMPPIController` перед каждым вызовом `GetNextNode` считывает файл и переводит мировые координаты в ячейки костмапа методом `nav2_costmap_2d::Costmap2D::worldToMap`:

$$row = \frac{y_w - o_y}{\Delta}, \quad col = \frac{x_w - o_x}{\Delta}, \quad v_{row} = \frac{v_y}{\Delta}, \quad v_{col} = \frac{v_x}{\Delta},$$

где Δ — разрешение костмапа (м/ячейку), (o_x, o_y) — координаты начала костмапа в мировом фрейме.

В каждом шаге роллаута t вычисляется предсказанное положение препятствия:

$$p_{row}(t) = row + v_{row} \cdot t \cdot \Delta t, \quad p_{col}(t) = col + v_{col} \cdot t \cdot \Delta t,$$

и добавляется штраф:

$$c_{obs}(t) = \begin{cases} c_{hard}, & d(t) < r_{hard}, \\ c_{soft} \left(1 - d(t)/r_{soft}\right)^2, & r_{hard} \leq d(t) < r_{soft}, \\ 0, & \text{иначе,} \end{cases}$$

где $d(t)$ — расстояние от позиции роллаута до предсказанного центра цилиндра.

В RA-MPPI тот же штраф применяется в обоих типах роллаутов: номинальных (для оценки средней стоимости) и возмущённых (для вычисления CVaR-хвоста), что позволяет оценивать риск столкновения с движущимся препятствием с учётом неопределённости управления.

Параметры. Жёсткий радиус $r_{hard} = (r_{cyl} + m_{hard})/\Delta$ ($r_{cyl} = 0,15$ м — радиус цилиндра, $m_{hard} = 0,09$ м — запас), мягкий $r_{soft} = 0,70$ м/ Δ .

5.6 Результаты экспериментов

Тестирование выполнено на 16 сценариях — декартово произведение четырёх комбинаций *планировщик-контроллер* и четырёх миров. Каждый сценарий запускался 10 раз независимо; на каждом прогоне поднимался независимый экземпляр Gazebo, Nav2 и процесса `move_cylinders`, bag-файл записывался полностью и анализировался скриптом `analyze_bag.py`. Фиксировалось число успешных и неудачных попыток, а метрические показатели усреднялись по успешным прогонам.

Сводные результаты приведены в таблице 5.1. Колонка *Планировщик* обозначает комбинацию глобального планировщика и локального контроллера (`astar/dstar` в паре с MPPI/RA-MPPI); колонка *Мир* — тестовый сценарий: `static` (без подвижных препятствий)

или `dynamic_N` (мир с N движущимися цилиндрами). Колонки *Усп.* и *Неуд.* содержат число успешных и неудачных прогонов из десяти; успешным считается прогон, в котором робот достиг окрестности цели с толерантностью 0,35 м за 120 с. Колонка *Откл.*, м — среднее расстояние от финальной точки робота до цели по успешным прогонам; *Путь*, м — средняя суммарная длина траектории по данным одометрии; *Время*, с — средняя длительность теста от первой до последней позы `/amcl_pose`. Колонка *Сбл.* приводит среднее число срабатываний `CollisionMonitor` (`action_type` $\neq 0$), разделённых паузой более 2 с, и отражает частоту принудительных торможений из-за динамических препятствий.

Таблица 5.1: Результаты сквозного тестирования навигационного стека ROS 2 / Nav2 (10 прогонов на сценарий).

Планировщик	Мир	Усп.	Неуд.	Откл., м	Путь, м	Время, с	Сбл.
A^* + MPPI	static	10	0	0,252	4,21	18,1	0,00
D^* Lite + MPPI	static	10	0	0,254	4,46	21,4	0,00
A^* + RA-MPPI	static	10	0	0,190	4,47	18,7	0,00
D^* Lite + RA-MPPI	static	10	0	0,245	4,45	16,6	0,00
A^* + MPPI	dynamic_1	10	0	0,240	4,25	19,3	0,00
D^* Lite + MPPI	dynamic_1	10	0	0,262	4,47	21,8	0,10
A^* + RA-MPPI	dynamic_1	10	0	0,180	4,62	22,6	0,00
D^* Lite + RA-MPPI	dynamic_1	10	0	0,237	4,49	18,0	0,00
A^* + MPPI	dynamic_5	9	1	0,238	4,78	30,7	0,30
D^* Lite + MPPI	dynamic_5	10	0	0,254	4,51	22,1	0,00
A^* + RA-MPPI	dynamic_5	10	0	0,194	4,91	22,2	0,00
D^* Lite + RA-MPPI	dynamic_5	10	0	0,221	4,58	19,4	0,00
A^* + MPPI	dynamic_10	10	0	0,221	4,84	41,9	1,00
D^* Lite + MPPI	dynamic_10	10	0	0,258	4,55	24,3	0,00
A^* + RA-MPPI	dynamic_10	10	0	0,148	5,26	27,3	0,20
D^* Lite + RA-MPPI	dynamic_10	9	1	0,209	4,90	23,6	0,22

В статическом мире все четыре конфигурации справляются со 100 % успешностью. Быстрейшим оказывается D^* Lite + RA-MPPI (16,6 с), A^* + MPPI и A^* + RA-MPPI сопоставимы (18,1 и 18,7 с), тогда как D^* Lite + MPPI немного медленнее (21,4 с). Длина пути варьируется в узком диапазоне 4,2–4,5 м, что подтверждает малозначимость выбора глобального планировщика для метрики пути в простых условиях.

При наличии одного движущегося цилиндра (`dynamic_1`) все конфигурации также достигают 100 % успешности; порядок скоростей сохраняется: D^* Lite + RA-MPPI лидирует (18,0 с), A^* + MPPI и A^* + RA-MPPI — 19,3 и 22,6 с, D^* Lite + MPPI — 21,8 с. Единичное срабатывание `CollisionMonitor` у D^* Lite + MPPI (0,10) отражает то, что инкрементальный планировщик реже уводит робота в обход, вынуждая локальный контроллер тормозить.

В сценарии `dynamic_5` впервые проявляется нестабильность A^* + MPPI: 9 успешных прогонов из 10, среднее время на успешных прогонах — 30,7 с, среднее число сближений — 0,30. Остальные три конфигурации сохраняют 100 % успешность при времени 19–22 с. Неудача A^* + MPPI воспроизводится стабильно (подтверждена повторными прогонами): при плотном расположении пяти цилиндров MPPI-контроллер с эвристикой A^* периодически попадает в тупиковое локальное оптимальное состояние, из которого не может выйти за 120 с.

В наиболее сложном сценарии `dynamic_10` контраст между конфигурациями максимален. $A^* + \text{MPPI}$ остаётся полностью успешным (10/10), однако ценой резкого роста времени (41,9 с) и высокого числа сближений (1,00 за прогон): контроллер многократно тормозит из-за движущихся цилиндров. $D^* \text{ Lite} + \text{MPPI}$ справляется значительно быстрее (24,3 с) при нулевых сближениях — инкрементальное перепланирование позволяет заблаговременно обойти динамические объекты. $A^* + \text{RA-MPPI}$ показывает 27,3 с при 0,20 сближениях: CVaR-штраф удерживает робота от опасной близости, однако без глобального перепланирования траектория длиннее. $D^* \text{ Lite} + \text{RA-MPPI}$ даёт 9/10 (23,6 с, 0,22 сближения): один неудачный прогон вызван блокировкой сразу несколькими цилиндрами, после чего навигация не укладывалась в таймаут.

Финальное отклонение от цели по успешным прогонам составляет 0,15–0,26 м во всех сценариях, что существенно меньше порога 0,35 м и свидетельствует о стабильной точности позиционирования независимо от сложности среды.

5.7 Выводы по главе

Эксперименты показали, что исследованные в главах 3–4 алгоритмы успешно работают в составе целевой инженерной системы — ROS 2 / Nav2 / Gazebo Sim. Сборка и стабилизация интегрированного стека потребовала разработки двух собственных пакетов (`dstar_lite_planner`, `ramppi_controller`), кинематического модуля перемещения препятствий и набора скриптов оркестрации тестов и пост-обработки. Из 16 сценариев 14 завершились со 100 % успешностью; единственные исключения — $A^* + \text{MPPI}$ в `dynamic_5` (9/10) и $D^* \text{ Lite} + \text{RA-MPPI}$ в `dynamic_10` (9/10) — отражают реальную сложность навигации при высокой плотности движущихся препятствий, а не дефекты реализации. Наиболее стабильной и быстрой во всех динамических мирах оказывается связка $D^* \text{ Lite} + \text{MPPI}$, сочетающая инкрементальное перепланирование с нулевыми сближениями при `dynamic_10`; $D^* \text{ Lite} + \text{RA-MPPI}$ демонстрирует наименьшее время в статичных и малодинамичных сценариях (16,6 и 18,0 с).

Заключение

В рамках курсовой работы решена комплексная задача исследования и практической реализации алгоритмов навигации мобильного робота в статической и динамической среде.

В аналитической части выполнен систематический обзор современных методов глобального и локального планирования. Рассмотрены графовые (Дейкстра, A^* , D^* , D^* Lite, LPA*, ARA*), сэмплирующие (RRT, RRT*, BIT*, RABIT*, Risk-DTRRT) и бионические (GA, ACO, ABC, PSO) глобальные планировщики, а также классические контроллеры (PID, LQR), геометрические методы (DWA, VO, APF, VFH) и оптимизационные подходы (MPC, MPPI, RA-MPPI и их производные). На основе сравнительного анализа обоснован выбор D^* Lite в качестве глобального планировщика, обеспечивающего инкрементальное перепланирование при локальных изменениях карты. На уровне локального управления для последующего экспериментального сравнения отобраны MPPI и RA-MPPI — стохастические предсказательные контроллеры, способные работать с нелинейной динамикой и учитывать препятствия через функцию стоимости.

Для экспериментального сравнения глобальных планировщиков разработана собственная программная модель на C++ и проведено сравнение A^* и D^* Lite на бенчмарке «Arena» (49×49 клеток) при 1000 случайных задачах на каждую конфигурацию. Показано, что в статической среде A^* остаётся оптимальным выбором благодаря лёгкому однократному поиску (0,12–0,42 мс против 0,7–3,2 мс у D^* Lite), однако в режиме высокой динамики (15 препятствий за ход) D^* Lite сокращает время обновления пути в 15–20 раз (3–10 мс против 150–200 мс), что делает его единственным масштабируемым решением для систем реального времени.

На той же модели проведено сравнение локальных контроллеров MPPI и RA-MPPI на 100 навигационных задачах. RA-MPPI реализует мягкий CVaR-штраф на основе возмущённых роллаутов: в режиме малой динамики (1 препятствие) оба алгоритма достигают одинаковой успешности (97 % у обоих), при высокой динамике (10 препятствий) — 100 % у MPPI против 99 % у RA-MPPI, различие в пределах статистической погрешности. Вычислительная стоимость обоих методов составляет 1,7 мс.

На этапе практической реализации алгоритмы интегрированы в навигационный стек ROS 2 / Nav2 / Gazebo Sim в виде двух собственных плагинов: `dstar_lite_planner` (глобальный) и `ramppi_controller` (локальный). Дополнительно разработаны модуль кинематического перемещения цилиндров и набор скриптов оркестрации экспериментов. На сквозном бенчмарке из 16 сценариев (4 пары планировщик–контроллер на 4 мирах с числом подвижных препятствий 0–10, по 10 прогонов на каждый сценарий) подтверждена совместная работоспособность всех конфигураций на физическом симуляторе TurtleBot3 Burger; 14 из 16 сценариев достигли 100 % успешности.

Полученные результаты подтверждают реализуемость гибридной архитектуры на собственных алгоритмах в составе индустриального стека Nav2 и могут служить основой для дальнейших исследований автономной навигации в непредсказуемой динамической среде и многороботных системах.

Список литературы

- [1] Kuanqi CAI, Chaoqun WANG, Jiyu CHENG, Shuang SONIG, SILVA Clarence W. DE и MENG Max Q.-H. «Mobile Robot Path Planning in Dynamic Environments: A Survey». В: *Instrumentation* 6.2 (2024), с. 90—100.
- [2] Karthik Karur, Nitin Sharma, Chinmay Dharmatti и Joshua E. Siegel. «A Survey of Path Planning Algorithms for Mobile Robots». В: *Vehicles* 3.3 (2021), с. 448—468.
- [3] Chengmin Zhou, Bingding Huang и Pasi Fränti. «A review of motion planning algorithms for intelligent robotics». В: *arXiv preprint, arXiv:2102.02376, version 2* (2021).
- [4] E. W. Dijkstra. «A Note on Two Problems in Connexion with Graphs». В: *Numerische Mathematik* 1 (1959), с. 269—271.
- [5] Peter Hart и Nils Nilsson. «A Formal Basis for the Heuristic Determination of Minimum Cost Paths». В: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), с. 100—107.
- [6] Jingjin Yu и Steven M. LaValle. «The Focussed D Algorithm for Real-Time Replanning». В: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI '95)* (1995), с. 1652—1659.
- [7] Sven Koenig и Maxim Likhachev. «D* Lite». В: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)* (2002), с. 476—483.
- [8] Maxim Likhachev, David I. Ferguson, Geoffrey J. Gordon, Anthony (Tony) Stentz и Sebastian Thrun. «Anytime Dynamic A: An Anytime, Replanning Algorithm». В: *Proceedings of the 15th International Conference on Automated Planning and Scheduling (ICAPS)* (2005), с. 262—271.
- [9] Liding Zhang, Kuanqi Cai, Zewei Sun, Zhenshan Bing, Chaoqun Wang, Luis Figueredo, Sami Haddadin и Alois Knoll. «Motion planning for robotics: A review for sampling-based planners». В: ().
- [10] Steven M. LaValle. «Rapidly-exploring Random Trees: A New Tool for Path Planning». В: *TR-98-11, Computer Science Dept., Iowa State University* (1998).
- [11] Sertac Karaman и Emilio Frazzoli. «Sampling-based Algorithms for Optimal Motion Planning». В: *The International Journal of Robotics Research* 30.7 (2011), с. 846—894.
- [12] Jonathan D. Gammell, Siddhartha S. Srinivasa и Timothy D. Barfoot. «Batch Informed Trees (BIT): Sampling-based optimal planning via the heuristically guided search». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (2015), с. 3067—3074.
- [13] Sanjiban Choudhury, Jonathan D. Gammell, Timothy D. Barfoot, Siddhartha S. Srinivasa и Sebastian Scherer. «Regionally Accelerated Batch Informed Trees (RABIT): A Framework to Integrate Local Information into Optimal Path Planning». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (2016), с. 4207—4214.

- [14] Wenzheng Chi, Chaoqun Wang, Jiankun Wang и Max Q.-H. Meng. «Risk-DTRRT-Based Optimal Motion Planning Algorithm for Mobile Robots». В: *IEEE Transactions on Automation Science and Engineering* 16.3 (2018/2019), с. 1271—1288.
- [15] Shiwei Lin, Jianguo Wang и Xiaoying Kong. «Bio-Inspired Reactive Approaches for Automated Guided Vehicle Path Planning: A Review». В: *Biomimetics* 11.1 (2026).
- [16] J. H Holland. *Adaptation in Natural and Artificial Systems*. Four volumes. University of Michigan Press, 1975.
- [17] Dorigo M., Maniezzo V. и Colorni A. «Ant System: Optimization by a Colony of Cooperating Agents». В: *IEEE Transactions on Systems, Man, and Cybernetics – Part B* 26.1 (1996), с. 29—41.
- [18] Karaboga D. и Basturk B. «A powerful and efficient algorithm for numerical function optimization: Artificial Bee Colony (ABC) algorithm». В: *Journal of Global Optimization* 39.3 (2007), с. 459—471.
- [19] Kennedy J. и Eberhart R. «Particle Swarm Optimization». В: *Proceedings of IEEE International Conference on Neural Networks (ICNN'95)* (1995), с. 1942—1948.
- [20] Alexandru Bârsan. «Position Control of a Mobile Robot through PID Controller». В: *Acta Universitatis Cibiniensis. Technical Series* 71 (2019), с. 14—20.
- [21] Zefan Su, Hanchen Yao, Jianwei Peng, Zhelin Liao, Zengwei Wang, Hui Yu, Houde Dai и Tim C. Lueth. «LQR-based control strategy for improving human–robot companionship and natural obstacle avoidance». В: *Biomimetic Intelligence and Robotics* 4.4 (2024).
- [22] Baiyu Tian, Tianliang Lin, Chunhui Zhang, Zhongshen Li, Shengjie Fu и Qihuai Chen. «Mixed Linear Quadratic Regulator Controller Design for Path Tracking Control of Autonomous Tracked Vehicles». В: *Lecture Notes in Mechanical Engineering* (2024), с. 127—136.
- [23] Chukwuma Samuel Okafor, Chimaihe B. Mbachu, Chidiebere N. Muoghalu и Bonaventure O. Ekengwu. «Positioning and Trajectory Tracking with Deflection Suppression in Flexible Link Robotic Manipulator Using PID-LQR Controller». В: *Asian Journal of Science, Technology, Engineering, and Art* 3.4 (2025), с. 1131—1146.
- [24] Dieter Fox, Wolfram Burgard и Sebastian Thrun. «The Dynamic Window Approach to Collision Avoidance». В: *IEEE Robotics & Automation Magazine* (1997).
- [25] Yanjie Cao и Norzalilah Mohamad Nor. «An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation». В: *Decision Analytics Journal* (2024).
- [26] Paolo Fiorini и Zvi Shiller. «Motion Planning in Dynamic Environments Using Velocity Obstacles». В: *The International Journal of Robotics Research* 17.7 (1998), с. 760—772.

- [27] Wanxin Liu, Bo Zhang, Pudong Liu, Jian Pan и Shiyu Chen. «Velocity Obstacle Guided Motion Planning Method in Dynamic Environments». В: *Journal of King Saud University – Computer and Information Sciences* 36.1 (2024).
- [28] Oussama Khatib. «Real-Time Obstacle Avoidance for Manipulators and Mobile Robots». В: *The International Journal of Robotics Research* 5.1 (1986), с. 90—98.
- [29] Shahid Mohammad Mulla, Aryan Kanakapudi, Lakshmi Narasimhan и Anuj Tiwari. «RF-Source Seeking with Obstacle Avoidance using Real-time Modified Artificial Potential Fields in Unknown Environments». В: *arXiv preprint, arXiv:2506.06811, version 1* (2025).
- [30] Johann Borenstein и Yoram Koren. «The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots». В: *IEEE Journal of Robotics and Automation* 7.3 (1991).
- [31] Andrej Babinec, Martin Dekan, František Duchoň и Anton Vitko. «Modifications of VFH Navigation Methods for Mobile Robots». В: *Procedia Engineering* 48 (2012), с. 10—14.
- [32] Shuyou Yu, Matthias Hirche, Yanjun Huang, Hong Chen и Frank Allgöwer. «Model predictive control for autonomous ground vehicles: a review». В: *Autonomous Intelligent Systems* 1.4 (2021).
- [33] Williams G., Aldrich A. и Theodorou E. A. «Model Predictive Path Integral Control: From Theory to Parallel Computation». В: *Journal of Guidance, Control, and Dynamics* 40.2 (2017), с. 344—357.
- [34] Guzman R., Oliveira R. и Ramos F. «Model predictive path integral control for car driving with autogenerated cost map based on prior map and camera image». В: *2019 IEEE Intelligent Transportation Systems Conference (ITSC)* (2019), с. 2109—2114.
- [35] Williams G., Drews P., Goldfain B., Rehg J. M. и Theodorou E. A. «Aggressive driving with model predictive path integral control». В: *IEEE International Conference on Robotics and Automation (ICRA)* (2016), с. 1433—1440.
- [36] Balci, Isin M., Bakolas, Efstathios, Vlahov, Bogdan I., Theodorou и Evangelos A. «Constrained Covariance Steering Based Tube-MPPI». В: *2022 American Control Conference (ACC)* (2022), с. 4197—4202.
- [37] Gandhi M. S., Vlahov B., Gibson J., Williams G. и Theodorou E. A. «Robust Model Predictive Path Integral Control: Analysis and Performance Guarantees». В: *IEEE Robotics and Automation Letters* 6.2 (2021), с. 1423—1430.
- [38] Yin Ji, Zhang Zheng и Tsiotras Panagiotis. «Risk-Aware Model Predictive Path Integral Control Using Conditional Value-at-Risk». В: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2023, с. 7937—7943.
- [39] Guzman R., Oliveira R. и Ramos F. «Adaptive Model Predictive Control by Learning Classifiers». В: *Proceedings of The 4th Annual Learning for Dynamics and Control Conference (PMLR)* 168 (2022), с. 480—491.